



Actividad práctica en parejas

Objetivo: reconocer en un código que funciona las cinco señales que describen los principios SOLID, refactorizarlo, y comprobar en el propio proyecto que una publicación nueva y un formato nuevo entran sin abrir la clase que los usa.

Los archivos

El código de la actividad está en el repositorio de la cátedra. Si ya está clonado, alcanza con actualizarlo desde la carpeta del repositorio:

```
git pull
```

Si no, se clona una única vez:

```
git clone https://github.com/P00-CR/poo-2026.git
```

También está en archivo comprimido en el classroom. La carpeta de hoy es 08-solid/actividad.

El dominio cambia. Es el catálogo de una biblioteca, con tres archivos. `Publicacion.java` es una sola clase que representa libros y revistas, y un atributo `tipo` dice cuál es cada una. `Biblioteca.java` guarda el catálogo, lo lista y lo exporta. `Programa.java` es el programa de prueba, que no se modifica salvo comentar y descomentar los bloques que indica cada parte.

Parte 1: ver el problema

Ejecutar `Programa` tal como viene. Leer `Publicacion.java` y `Biblioteca.java` completos. Responder por escrito.

1. La biblioteca quiere sumar tesis al catálogo. Una tesis se presta a domicilio, la multa por retraso es de \$800 por día y lleva el nombre de la universidad. Listar cada lugar de `Publicacion.java` que habría que tocar para que entre.
2. `revista.prestar("Ana")` compila, y devuelve `false`. ¿Qué es lo que hoy impide que una revista salga de la sala, el compilador o un `if`? ¿Qué pasa el día que se agrega un tipo nuevo que tampoco se presta y nadie se acuerda de ese `if`?
3. El contador pide el catálogo separado por comas en vez de JSON. ¿Qué archivo hay que abrir para hacerlo? ¿Qué tiene que ver ese archivo con los formatos de exportación?
4. Contar cuántas responsabilidades distintas tiene `Publicacion`, y quién pediría cada una.
5. Los problemas de las preguntas 1 a 4 tienen nombre en la pre clase. Indicar qué principio describe cada uno. Puede haber más de uno por pregunta, y el mismo principio puede repetirse.

Parte 2: refactorizar

Los pasos están marcados como `TODO` en `Publicacion.java` y `Biblioteca.java`. En orden.



- TODO 1: Publicacion pasa a ser abstracta, sin tipo, paginas ni numero. Se queda con título, autor y año, sus getters y un mostrarDetalle() concreto con la parte común. Los métodos de préstamo, la multa y exportarJson() se van de ahí.
- TODO 2: escribir Libro.java (con páginas) y Revista.java (con número), las dos heredando de Publicacion y sobrescribiendo mostrarDetalle() con super.mostrarDetalle() adentro.
- TODO 3: escribir la interfaz Prestable.java, con prestar(String socio), devolver() y multaPorRetraso(int diasDeRetraso). La implementa Libro, que guarda a quién está prestado. Revista no la implementa.
- TODO 4: escribir la interfaz Exportador.java, con un único método String exportar(Publicacion publicacion), y ExportadorJson.java que la implementa con el formato que hoy está en Publicacion. Biblioteca pasa a recibir un Exportador por constructor y a usarlo en exportarCatalogo().

Es esperable que el proyecto no compile desde el TODO 1 hasta terminar el TODO 4, porque Biblioteca y Programa siguen usando lo que se está sacando. No hace falta ir arreglando esos errores uno por uno, se resuelven solos al terminar el TODO 4 y cambiar los bloques de Programa.

En Programa.java, comentar el bloque de la parte 1 entero y descomentar el de la parte 2. Ejecutar. La salida tiene que ser la misma que antes, salvo las dos líneas de la revista en préstamos, que ya no existen, y el campo tipo del JSON.

Hay una línea comentada, revista.prestar("Ana"). Descomentarla y leer lo que dice el compilador. Antes de volver a comentarla, responder por escrito cuál de las dos respuestas es mejor para la biblioteca, que prestar devuelva false o que la línea no compile, y por qué.

Parte 3: probar

- TODO 5: escribir Tesis.java. Hereda de Publicacion, implementa Prestable, cobra \$800 por día de retraso y lleva la universidad. Dentro del bloque de la parte 2 hay cuatro líneas de la tesis comentadas con // y marcadas con TODO 5. Descomentarlas y ejecutar. La tesis tiene que aparecer en el catálogo, en el conteo, en los préstamos y en la exportación.
- TODO 6: escribir ExportadorCsv.java, que devuelve título, autor y año separados por punto y coma. Cambiar la primera línea del bloque, marcada con TODO 6, para que Biblioteca lo reciba, y ejecutar.

Antes de seguir, contar cuántas líneas de Biblioteca.java, Libro.java, Revista.java y ExportadorJson.java hubo que modificar en la parte 3, y comparar con la lista de la pregunta 1.

Para discutir en la pareja y anotar:

- Libro y Tesis repiten prestar() y devolver() línea por línea. ¿Dónde iría ese código para escribirlo una sola vez, y qué relación tendrían Libro y Tesis con esa clase nueva? Con esa clase en el medio, ¿seguiría haciendo falta la interfaz Prestable?



- El `new ExportadorCsv()` lo escribe `Programa`. ¿Por qué no lo escribe `Biblioteca`, que es la que lo usa?